DOCKER

Docker vs. Kubernetes: Container Solutions Comparison

James Walker
Updated 17 Mar 2025 · 15 min read

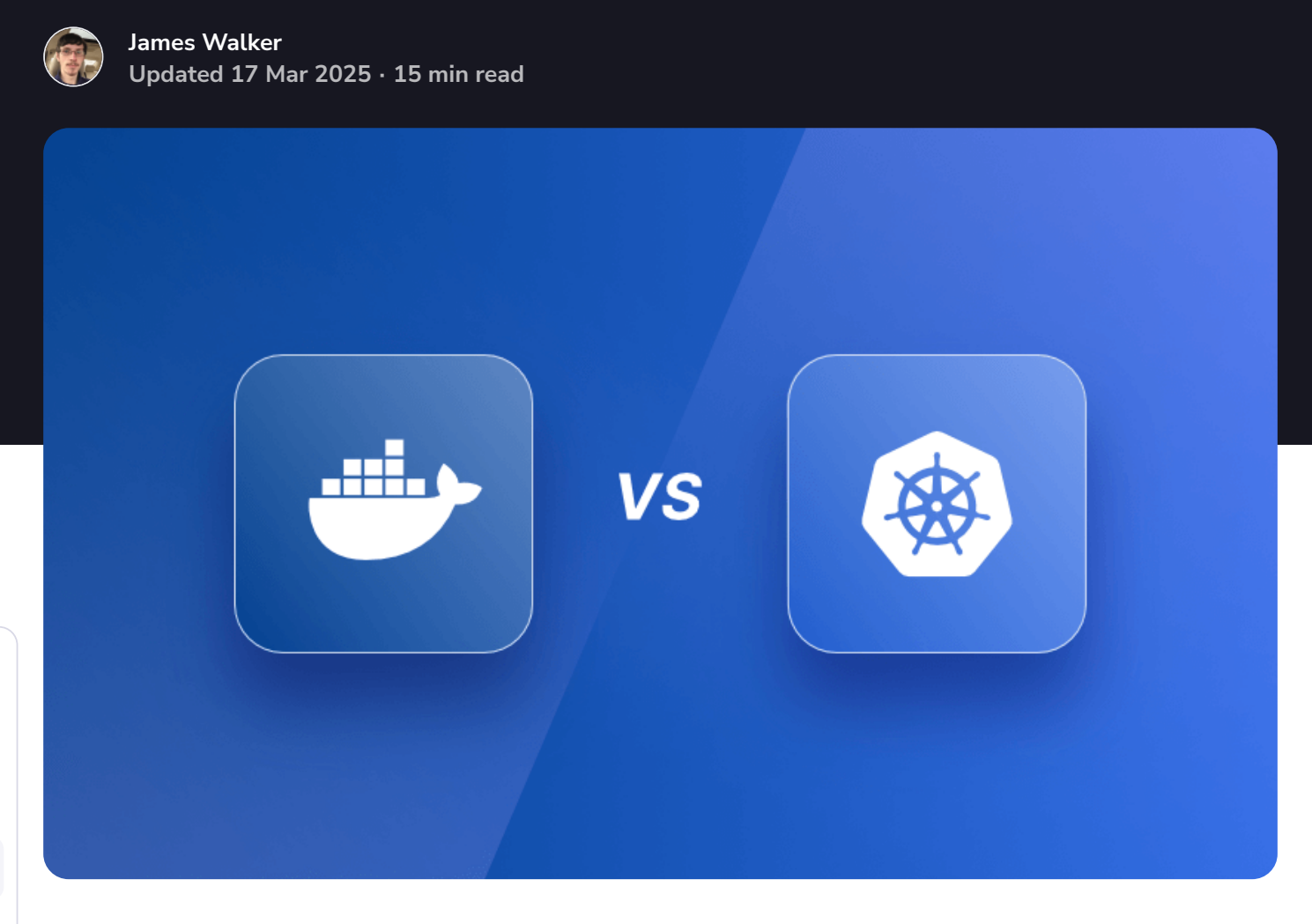


TABLE OF CONTENTS
What is Docker?
What is Kubernetes?
Docker vs Kubernetes: Similarities
Differences between Docker and Kubernetes
Is Kubernetes required for Docker?
Which is better: Docker or Kubernetes?
Using Docker with Kubernetes
Key points

🚀 Level Up Your Infrastructure Skills

You focus on building. We'll keep you updated. Get curated infrastructure insights that help you make smarter decisions.

Your email address

Subscribe

Docker and Kubernetes are two of the most popular container tools used for modern DevOps. Docker is a containerization platform for building and running container images, while Kubernetes is a container orchestration tool designed for deploying, scaling, and managing containers in production.

Docker and Kubernetes overlap in several ways but also have unique differences that affect the situations where they're used. You can use one tool without the other or allow them to complement each other by combining both in your workflows.

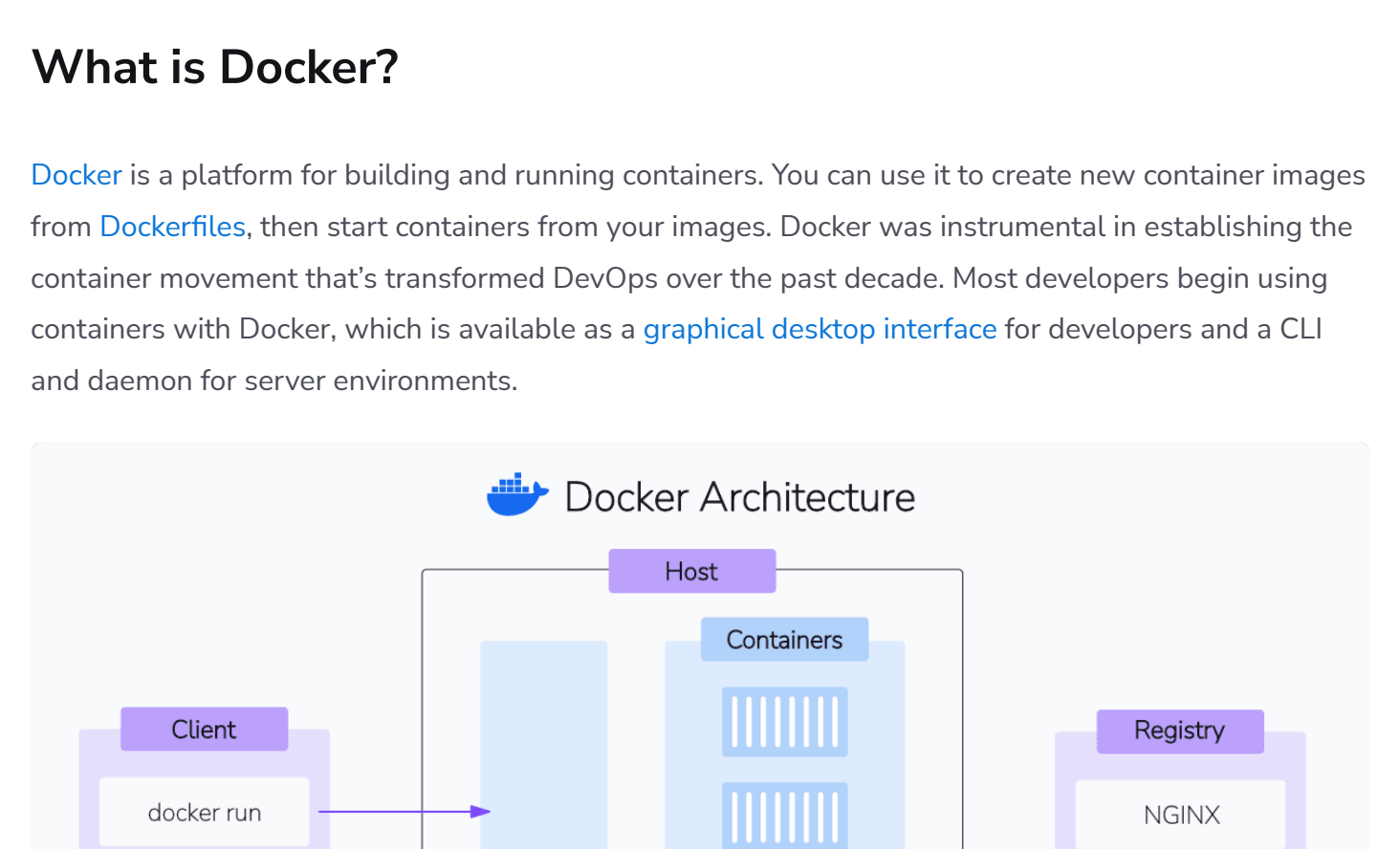
In this article, we'll explore their similarities and differences and then discuss the advantages of using each tool.

We will cover:

1. What is Docker?
2. What is Kubernetes?
3. Docker vs Kubernetes: Similarities
4. Differences between Docker and Kubernetes
5. Is Kubernetes required for Docker?
6. Which is better: Docker or Kubernetes?
7. Using Docker with Kubernetes

What is Docker?

Docker is a platform for building and running containers. You can use it to create new container images from Dockerfiles, then start containers from your images. Docker was instrumental in establishing the container movement that's transformed DevOps over the past decade. Most developers begin using containers with Docker, which is available as a graphical desktop interface for developers and a CLI and daemon for server environments.



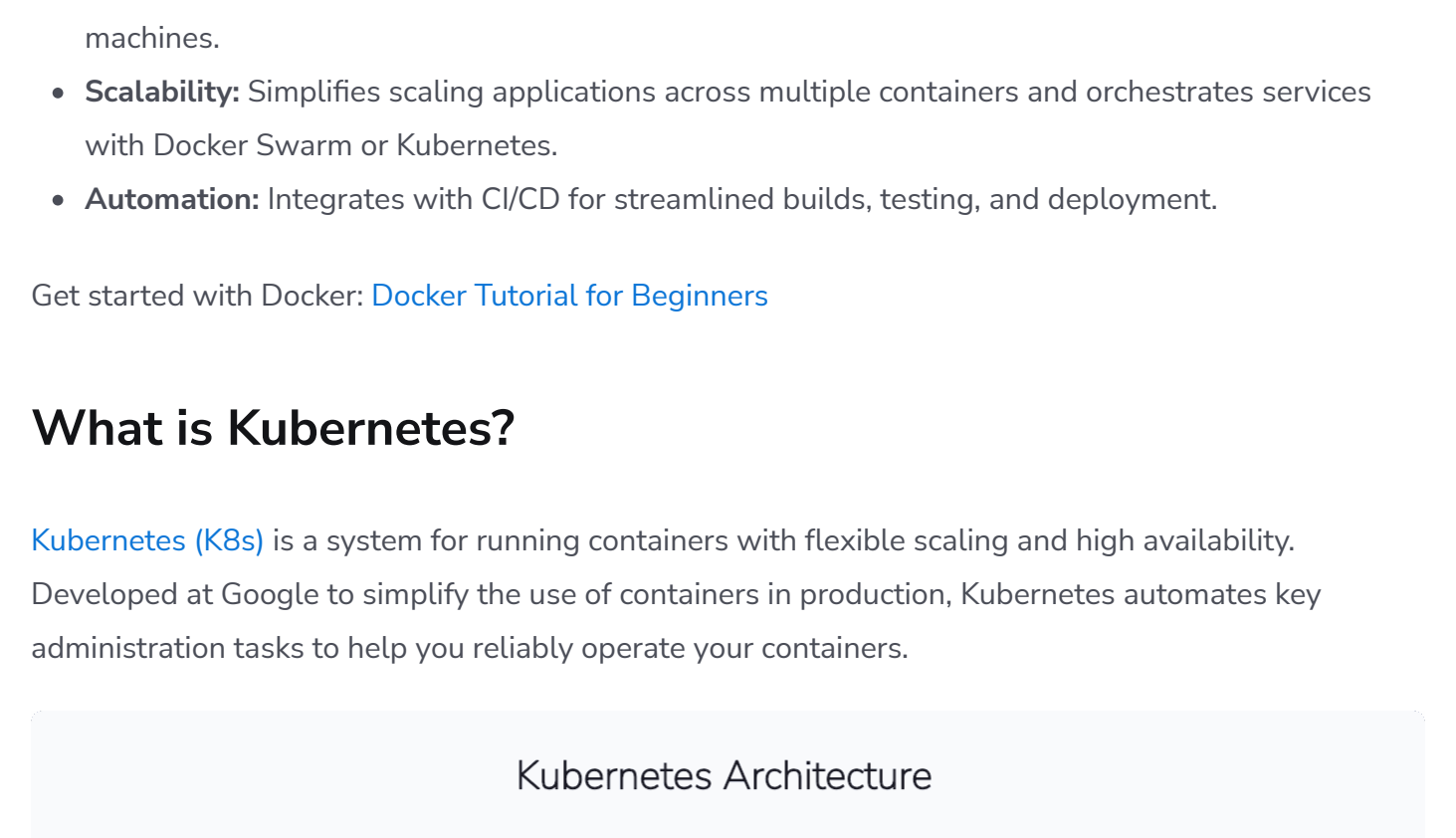
Key features of Docker:

- **Containerization:** Packages applications and their dependencies into isolated containers, ensuring consistent behavior across environments.
- **Portability:** Runs containers on any system with Docker installed, ensuring cross-platform compatibility.
- **Lightweight:** Uses minimal resources by sharing the host OS kernel instead of running full virtual machines.
- **Scalability:** Simplifies scaling applications across multiple containers and orchestrates services with Docker Swarm or Kubernetes.
- **Automation:** Integrates with CI/CD for streamlined builds, testing, and deployment.

Get started with Docker: Docker Tutorial for Beginners

What is Kubernetes?

Kubernetes (K8s) is a system for running containers with flexible scaling and high availability. Developed at Google to simplify the use of containers in production, Kubernetes automates key administration tasks to help you reliably operate your containers.



Several Kubernetes distributions are available. Compact versions like Minikube are ideal for local use, while managed services such as Google GKE and Amazon EKS allow you to rapidly provision new cloud-based clusters.

Key features of Kubernetes:

- **Orchestration:** Manages deployment, scaling, and operations of containerized applications automatically.
- **Load balancing:** Distributes network traffic across containers to maintain service reliability.
- **Self-healing:** Detects and restarts failed containers, replacing or rescheduling them as needed.
- **Automated rollouts and rollbacks:** Manages application updates with control over deployment and rollback.
- **Scalability:** Adjusts the number of running containers based on demand, supporting efficient resource use.

Get started with Kubernetes: Kubernetes Tutorial for Beginners

Docker vs Kubernetes: Similarities

Docker and Kubernetes each revolve around containers. You can use them to run containers, link them together, attach persistent storage, and provide network services.

Both Docker and Kubernetes are compatible with all OCI container images, runtimes, and container registries. Images created with Docker, for example, can be used in Kubernetes.

The two tools include their own networking models and storage abstractions. You can also use them to replicate container instances across multiple hosts–this is fundamental in Kubernetes, whereas Docker offers it as part of its optional Swarm mode.

💡 **You might also like:**

- Top Most Useful CI/CD Tools for DevOps
- Terraform vs. Kubernetes Comparison
- 15 Kubernetes Best Practices to Follow

Differences between Docker and Kubernetes

While Docker and Kubernetes share some similarities in containerization and application management, it's important to remember that Docker is primarily a platform for building and running containers, whereas Kubernetes is a container orchestration system that can operate on Docker containers or other container runtimes.

Beyond their high-level support for running containers, Docker and Kubernetes each possess several differences that lend them advantages and drawbacks in specific situations.

Table comparison

Here's how Docker and Kubernetes compare across some key characteristics:

	Docker	Kubernetes
Type of tool	Containerization platform	Container orchestrator
Lifecycle management	Basic container lifecycle management (e.g., restart containers after failure)	Complete container lifecycle management (proactive monitoring to ensure applications remain accessible after a host fails)
Abstraction level	Minimal abstraction – users interact directly with containers	Multiple abstraction layers are included; users interact with Kubernetes objects that model application components
Imperative/Declarative	Imperative Docker CLI commands or declarative configuration with Docker Compose.	Both supported. Declarative is preferred and used in most examples.
Scaling model	Swarm mode is required to replicate and scale containers. No auto-scaling support.	Built-in scaling support with configurable automatic horizontal auto-scaling.
Use cases	Developer use, CI/CD pipelines, and deployment of small applications in manually managed environments.	Development, test, CI/CD, and production environments with high availability and automatically provisioned infrastructure.
Observability	Basic container resource monitoring and logging.	Resource monitoring and logging, supported by easy integration with observability stacks such as Prometheus/Grafana.

1. Containerization

Docker and Kubernetes are both container tools, but they focus on different areas within the overall containerization space. Docker provides functionality for building and running containers, whereas Kubernetes is focused on their deployment and operation.

Docker includes components such as BuildKit for writing Dockerfiles and building images from them. You can also search Docker Hub to find images relevant to your workload, and access built-in tools for image security scans and SBOM generation.

Kubernetes only works with existing images and provides fewer features for managing them. You can run services inside your cluster that offer functions similar to those bundled with Docker, but they're not part of the core Kubernetes experience. The power of Kubernetes is its ability to scale containers across multiple physical Nodes, then ensure they remain accessible after Node failures.

2. Orchestration and scaling

Docker generally works with individual containers. In practice, most applications are formed from multiple services that need to be managed as a collective stack. Docker ecosystem tools, including Docker Compose and Swarm Mode, facilitate this, but they're relatively narrow in scope.

Kubernetes, meanwhile, is a purpose-built orchestrator for scaling containers across multiple nodes. It includes a robust set of features for assigning containers to nodes, maintaining resource capacities, and adjusting scaling configuration in response to actual demand. Although the initial setup procedure is more demanding, Kubernetes is better suited to the demands of real applications in live environments.

Another benefit of Kubernetes is its declarative-first design. Deployments are usually configured as YAML manifests, which describe your desired state. When you apply a manifest, Kubernetes automatically reconciles the changes and applies any actions required to transition your cluster to the new state.

By contrast, most users interact with Docker using imperative CLI commands; Compose and Swarm mode are required for declarative configuration of replicated services, but the result is still more limited than Kubernetes.

3. Networking

Docker and Kubernetes differ significantly in their approach to managing containerized applications beyond just containerization and orchestration. One key distinction is their networking models.

Docker's networking model is relatively straightforward, allowing containers on the same host to communicate using bridge, overlay, or host networks. Kubernetes, however, is built for large-scale distributed systems. It provides a flat networking model where pods can seamlessly communicate across nodes without NAT, making it more suitable for complex, multi-node deployments.

Kubernetes also supports advanced networking policies and service discovery, which are essential for handling microservices at scale.

4. Security

Security is another area where these two technologies diverge. Docker enhances the security of individual containers through a combination of features such as user namespaces for isolation, Secure Computing Mode (seccomp) profiles to restrict system calls, and image signing to ensure the integrity and authenticity of container images.

Kubernetes, on the other hand, provides security at a broader level with Role-Based Access Control (RBAC), network policies, and Pod Security Admission (PSA), which replaces the deprecated Pod Security Policies (PSP). It also integrates with enterprise authentication systems, making it a better choice for organizations that need strict access control and compliance.

While Docker secures the building blocks, Kubernetes fortifies the entire architecture, ensuring a more robust and scalable environment.

5. Abstraction level

Docker is a relatively low-level tool. It directly manages containers, and you need to use other tools, such as Compose, to link them together.

Kubernetes takes a higher-level view that includes more abstraction layers. Concepts such as Pods, ReplicaSets, and Deployments present a learning curve initially but allow more accurate and powerful modeling of your application.

Kubernetes can manage your supporting infrastructure too. It can provision networking components such as load balancers and storage volumes for you, using the resources available from your cloud provider. Abstracting away the differences between different clouds gives you more flexibility when deploying your workloads.

6. Ops features

Docker comes with extremely minimal ops features for overseeing your deployments. You can get basic resource utilization information from docker stats, or use docker logs to access a container's log stream. Beyond these features, there are no other observability capabilities, nor are there mechanisms for controlling access to Docker itself. This simplifies local use on your own machine but is restrictive in production.

Kubernetes comes with essential operations controls built-in. It has extensive support for Role-Based Access Control (RBAC), for example, which allows you to control the operations different users can perform. Kubernetes is also easily integrated with advanced observability solutions such as Prometheus, which can run in your cluster while simultaneously scraping metrics from it.

In summary, Kubernetes is designed around the requirements of operators, as it originated from this field. Docker is positioned as a developer-oriented tool, which means it lacks robust ops-specific features.

7. Use cases

Docker and Kubernetes serve different but complementary roles in containerized application management.

Docker is primarily a containerization platform that allows developers to create, package, and run applications in isolated environments. It is ideal for building, testing, and running applications on a single system or across small-scale deployments.

Kubernetes is designed to manage large-scale containerized applications across multiple nodes. It automates tasks like scaling, load balancing, and self-healing, making it suitable for managing production-level, distributed applications.

While Docker is well-suited for single-node setups, Kubernetes shines when handling complex workloads requiring automation. In practice, they are **often used together**, with Docker creating containers and Kubernetes orchestrating them across a distributed infrastructure.

8. Learning curve

Docker has a lower learning curve compared to Kubernetes because it focuses on containerization, making it easier for developers to package and run applications. It primarily involves learning basic commands for building, running, and managing containers.

So, for beginners, Docker is the easier starting point, while Kubernetes requires foundational knowledge of containers before diving into its orchestration capabilities. However, mastering Kubernetes is essential for scaling and managing containerized applications in production environments.

Is Kubernetes required for Docker?

Images you build with Docker can be deployed to a Kubernetes cluster. The two tools operate independently, however: **Kubernetes does not directly depend on Docker.** You don't need Docker installed to use Kubernetes, and vice versa.

Both Docker and Kubernetes are compliant with the Open Container Initiative (OCI) specifications. This project defines standards for interoperability between different container runtimes and image formats. Docker outputs OCI-compatible images, which can be run by any OCI runtime. The default runtime used by Docker and Kubernetes is containerd, but alternatives such as CRI-O are available.

It's worth noting that Kubernetes *did* historically use Docker to run the containers in your cluster. A component called Dockershim provided an interface between Docker and Kubernetes. Dockershim was removed in Kubernetes v1.24; an OCI-compatible runtime must now be used instead.

Which is better: Docker or Kubernetes?

> Docker provides a streamlined way to develop and manage containers on a single host, which is particularly useful for local development and testing environments. Kubernetes, on the other hand, is designed for orchestrating and scaling containers across clusters, making it ideal for production and complex distributed applications.

Docker's simplicity means it's often the most convenient option during development. It's ideal for smaller applications with few components. Docker also has the advantage of familiarity and a low learning curve: most developers quickly become proficient at building images and starting containers.

Kubernetes is more complex, but it's also much more flexible and better suited to the requirements of operators running containers in production. Its support for scalability, high availability, container lifecycle management, and observability can't be replicated by Docker.

Additionally, the ability to quickly provision new Kubernetes clusters using managed cloud services can significantly accelerate your route to production.

In the end, there's no definitive answer to this question, as it depends on the container features you require and how they will be used.

See also: Docker Compose vs Kubernetes comparison.

Using Docker with Kubernetes

Nowadays, it's common for development teams to deploy both technologies as part of their container workflows, allowing them to complement each other: Docker handles individual containers, while Kubernetes manages clusters of them.

One major benefit of using Docker with Kubernetes is the ability to standardize and streamline the deployment process. Docker packages applications into isolated, consistent containers, ensuring that they run the same way across different environments. Kubernetes then orchestrates these containers, automating deployment, scaling, and management across a cluster of machines.

This combination enables high availability, efficient resource utilization, and simplified scaling, as Kubernetes can dynamically manage workloads and distribute traffic based on demand.

Together, Docker and Kubernetes streamline the process of deploying and maintaining applications at scale, making them ideal for microservices architectures and cloud-native applications.

Use cases

Let's consider some use case examples for using Kubernetes and Docker together:

- **CI/CD pipelines:** In a DevOps pipeline, Docker containers run application builds, tests, and deployments. Kubernetes manages the deployment, automating updates and rolling back versions in case of errors.
- **Big data and analytics:** A data processing pipeline using tools like Apache Spark can run on Docker containers managed by Kubernetes. Kubernetes handles resource scaling to meet data workload demands, enhancing performance without manual intervention.
- **Machine learning model deployment:** Machine learning models often require specific resources and can benefit from scalability. A machine learning model for image recognition is packaged in a Docker container and deployed to Kubernetes. Kubernetes scales the model based on API demand, allowing rapid response to spikes in usage, especially during peak hours.

Key points

Both Docker and Kubernetes are great tools for working with containers in DevOps situations. They're both capable of running containers, but they're typically used in different parts of the development workflow.

Docker is focused on application components. It's useful for building your container images and running them locally as you work. However, Docker lacks the operations features essential to the use of containers in production.

Kubernetes accommodates this use case by providing a platform for automating container deployments and lifecycle management. It helps you operate containers by providing features for dynamic scaling and robust fault tolerance.

Kubernetes is also easy to integrate with IaC and CI/CD solutions. You can use Spacelift to provision Kubernetes clusters with Terraform, for example, allowing you to automate infrastructure configuration. You can use Kubernetes to run all your container workloads across production, development, and CI/CD jobs, but might still want to choose Docker for quick testing on your laptop.

The most flexible CI/CD automation tool

Spacelift is an alternative to using homegrown solutions on top of a generic CI. It helps overcome common state management issues and adds several must-have capabilities for infrastructure management.

Start free trial

Written by

James Walker

James Walker is the founder of Heron Web, a UK-based software development studio providing bespoke solutions for SMEs. He has experience managing complete end-to-end web development workflows with DevOps, CI/CD, Docker, and Kubernetes. James is also a technical writer and has written extensively about the software development lifecycle, current industry trends, and DevOps concepts and technologies.

jhwalker.net

Read also

DOCKER 13 min read
Docker Swarm vs. Kubernetes – Key Differences Explained

DOCKER 14 min read
Top 12 Most Useful Docker Alternatives for 2025 [List]

KUBERNETES 13 min read
15 Common Kubernetes Pitfalls & Challenges

© 2025 Spacelift, Inc. All rights reserved